

GitHub Actions — CI/CD Notes

Based on: krishnaik06 GitHub Actions video (github.com/krishnaik06/appgithubaction)

1. What is CI/CD and GitHub Actions

Continuous Integration (CI)

Practice of automatically building and testing code every time someone pushes changes or opens a pull request. Catches bugs early, before they reach production. Example: every PR triggers a test suite run automatically.

Continuous Delivery / Deployment (CD)

Automatically deploying code that passes CI checks to staging or production. Removes manual "click deploy" steps.

GitHub Actions

- GitHub's built-in CI/CD platform — no third-party tool (like Jenkins) needed.
- Event-driven: workflows run in response to repo events (push, PR, issue created, schedule/cron, manual trigger, etc.).
- Not limited to DevOps — e.g. auto-labeling issues, auto-closing stale PRs, sending notifications.
- Runs on GitHub-hosted runners (Linux/Windows/macOS VMs) or self-hosted runners (your own machine/cloud instance).

Core building blocks (light in most intro videos, added here):

Concept	What it means
Workflow	A YAML file in <code>.github/workflows/</code> describing automation
Event	What triggers the workflow (push, pull_request, schedule, workflow_dispatch)
Job	A set of steps that run on one runner; jobs can run in parallel or depend on each other
Step	A single task inside a job — a shell command or a reusable Action
Action	A packaged, reusable unit (e.g. <code>actions/checkout@v4</code>) — like a function call
Runner	The VM/container that actually executes the job

Minimal working example:

```
name: CI Pipeline
on: [push, pull_request]
```

```

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4          # pulls your repo onto the runner

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'

      - name: Install dependencies
        run: pip install -r requirements.txt

      - name: Run tests
        run: pytest

```

Line-by-line logic: **on** defines triggers → **jobs.build.runs-on** picks the VM image → each step either calls a prebuilt **uses**: action or runs a raw shell command via **run**:.

2. Developer Workflow With Examples

This section of the video walks through how a real team uses GitHub Actions day-to-day. Typical pattern:

1. Developer creates a feature branch, pushes commits.
2. Opens a PR → triggers a workflow on the `pull_request` event.
3. Workflow runs lint + tests automatically, posts status checks on the PR (green check / red X).
4. Once merged to main, a second workflow triggers on push to main → builds a Docker image, deploys to staging/production.

Common triggers used in real projects:

```

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
  schedule:
    - cron: '0 6 * * *'          # runs daily at 6 AM UTC (e.g. your AQI predictor's schedule)
  workflow_dispatch:             # adds a manual "Run workflow" button in GitHub UI

```

Environment variables & secrets (usually glossed over, but essential):

```

steps:
  - name: Use secret API key
    env:
      API_KEY: ${ secrets.MY_API_KEY }
    run: python fetch_data.py

```

Secrets are stored in repo Settings → Secrets and Variables → Actions, never hardcoded — this is how you'd securely pass something like an Open-Meteo or Hopsworks API key.

Matrix builds (testing across multiple versions at once):

```

strategy:

```

```

matrix:
  python-version: ['3.9', '3.10', '3.11']
steps:
  - uses: actions/setup-python@v5
    with:
      python-version: ${ matrix.python-version }

```

This spins up 3 parallel jobs, one per Python version, instead of writing 3 separate job blocks.

3. Unit Testing Workflow Implementation

This is the hands-on segment where pytest (or similar) gets wired into a workflow that runs on every push/PR.

Typical structure:

```

name: Unit Tests

on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with:
          python-version: '3.11'

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt

      - name: Run unit tests
        run: pytest tests/ --cov=src --cov-report=xml

      - name: Upload coverage report
        uses: actions/upload-artifact@v4
        with:
          name: coverage-report
          path: coverage.xml

```

What's happening conceptually:

- checkout — without this, the runner is a blank VM with no access to your code.
- setup-python — installs the interpreter version you need, since runners aren't pre-configured for your stack.
- Dependency install — same as local dev, just automated.
- pytest with --cov — runs the test suite and measures code coverage.
- upload-artifact — saves the coverage report so it can be downloaded from the Actions tab later, since the runner's filesystem is destroyed after the job ends.

Subtlety worth knowing: if any step's command exits with a non-zero status (e.g. a failing test), the whole job is marked failed and later steps are skipped by default — that failure is what shows up as the red X on a PR.

Status badges (nice-to-know, often shown at the end of such videos):

`![CI](https://github.com/username/repo/actions/workflows/tests.yml/badge.svg)`

Drops a live pass/fail badge into your README.